AWS CLOUD INFRASTRUCTURE IMPLEMENTATION REPORT

S3-Triggered Lambda Image Processing and Recursive Invocation Prevention

1. AWS Lambda – Image Processing Layer

AWS Lambda is used to process images uploaded to Amazon S3. The Lambda function, `ImageThumbnailProcessor`, is triggered when a supported image is uploaded to the input bucket. The function uses the Python Pillow library to resize the image to a maximum dimension of 200 × 200 pixels and stores the generated thumbnail in a separate S3 output bucket.

Function configuration:

Parameter	Configuration
Lambda function	`ImageThumbnailProcessor`
Runtime	Python 3.14
Architecture	ARM64
Memory	512 MB
Timeout	30 seconds
Processing library	Pillow
Output format	Same supported image format as input

The function processes JPG, JPEG and PNG images. Unsupported files are skipped.

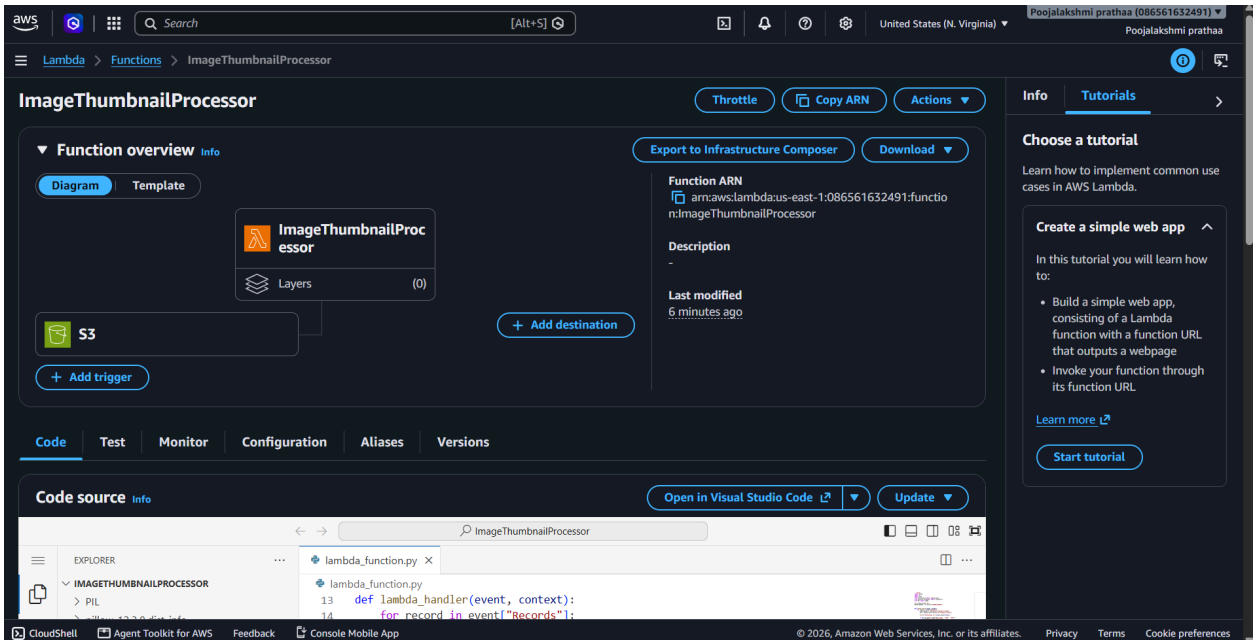


Figure 1. AWS Lambda function configuration for image processing.

2. Amazon S3 – Input and Output Storage

Amazon S3 provides scalable object storage for the image-processing workflow. Two separate buckets are used to avoid recursive invocation caused by output objects triggering the same Lambda function.

Resource	Configuration	Purpose
Input bucket	`poojalakshmi-image-input-2026`	Stores uploaded images
Input prefix	`incoming/`	Identifies objects eligible for processing
Output bucket	`poojalakshmi-image-output-2026`	Stores generated thumbnails
Output prefix	`thumbnails/`	Organizes thumbnail files

When a supported image is uploaded under `incoming/`, an S3 ObjectCreated event invokes Lambda. The resulting thumbnail is stored under `thumbnails/` in the output bucket. Since the output bucket is separate, the generated thumbnail does not retrigger the input bucket's notification.

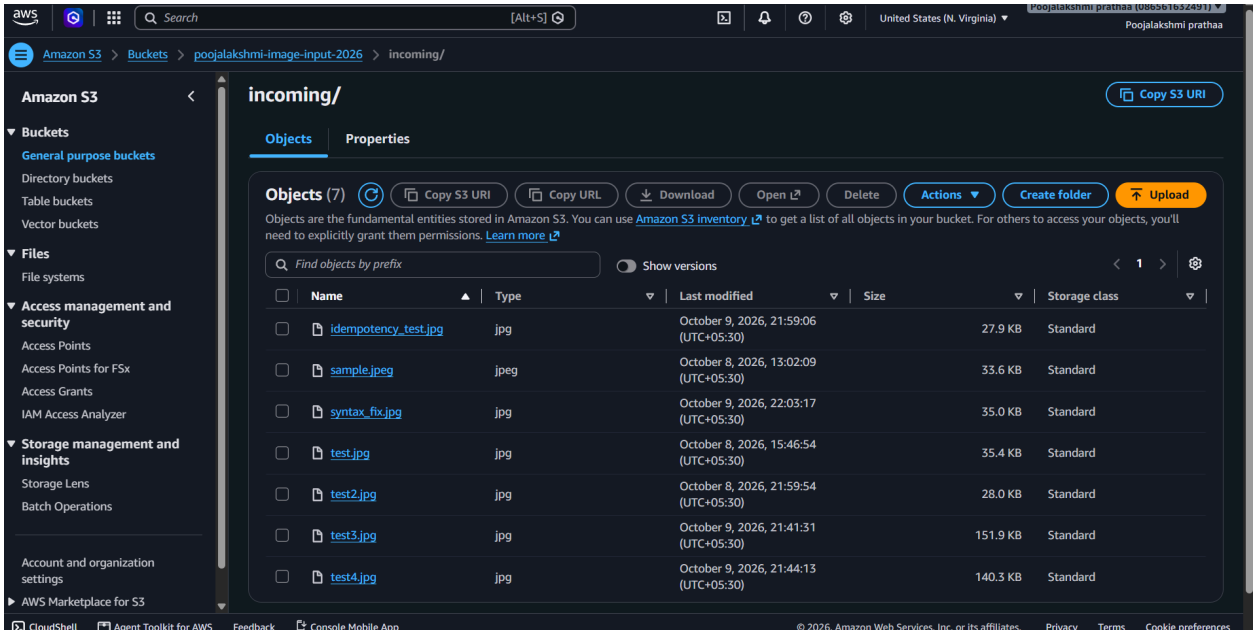


Figure 2. Input S3 bucket showing the `incoming/` folder and uploaded test images

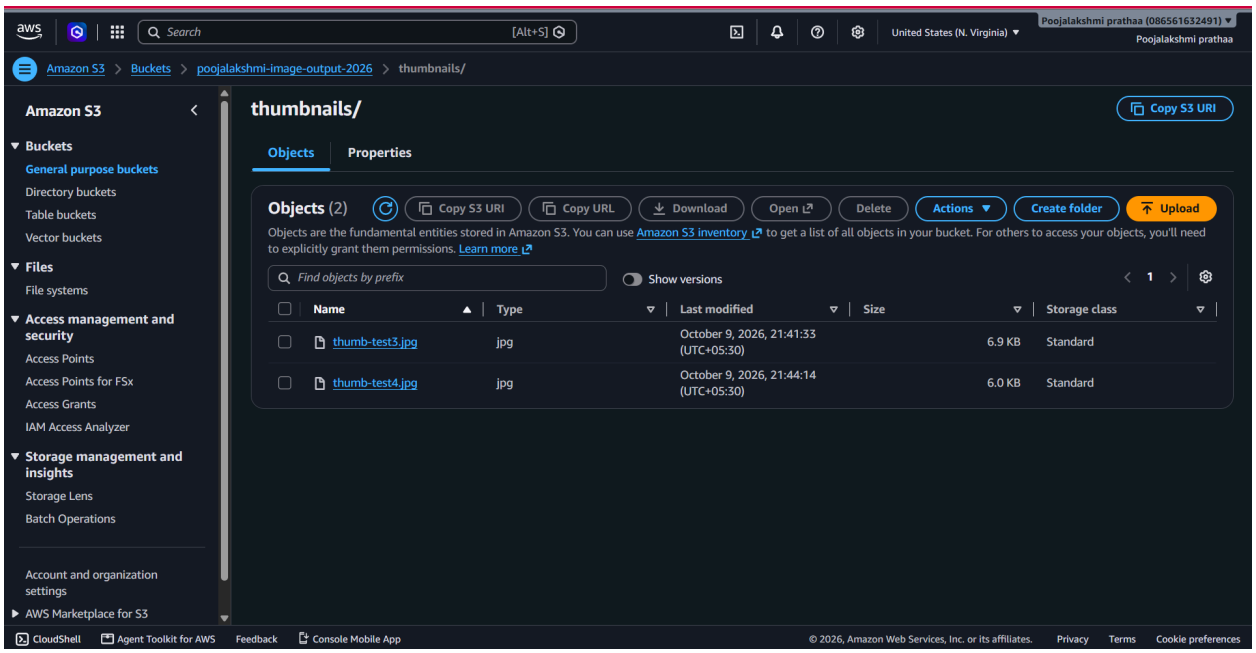


Figure 3. Output S3 bucket showing generated thumbnail files under `thumbnails/`.

3. Amazon S3 Event Notification and Trigger Configuration

An S3 event notification connects the input bucket to the Lambda function. The notification responds to object creation events and uses the `incoming/` prefix filter to restrict processing to the intended input objects.

The prefix filter reduces unnecessary invocations and helps prevent recursive processing. The output bucket is not configured as a trigger source for this function.

Figure 4. Lambda function overview showing the connected S3 trigger.

Figure 5. S3 trigger configuration showing the input bucket, ObjectCreated event and `incoming/` prefix filter.

4. AWS IAM – Identity and Access Management

IAM provides secure, controlled access to AWS resources. The Lambda function uses the execution role `ImageProcessorLambdaRole`, which has the `AWSLambdaBasicExecutionRole` managed policy and the custom `ImageProcessorS3Policy`.

The custom policy grants `s3:GetObject` access to input images under the `incoming/` prefix and `s3:PutObject` access to thumbnails under the output bucket's `thumbnails/` prefix. The managed execution policy provides standard CloudWatch Logs permissions.

This configuration follows the principle of least privilege by restricting S3 access to the resources required by the application.

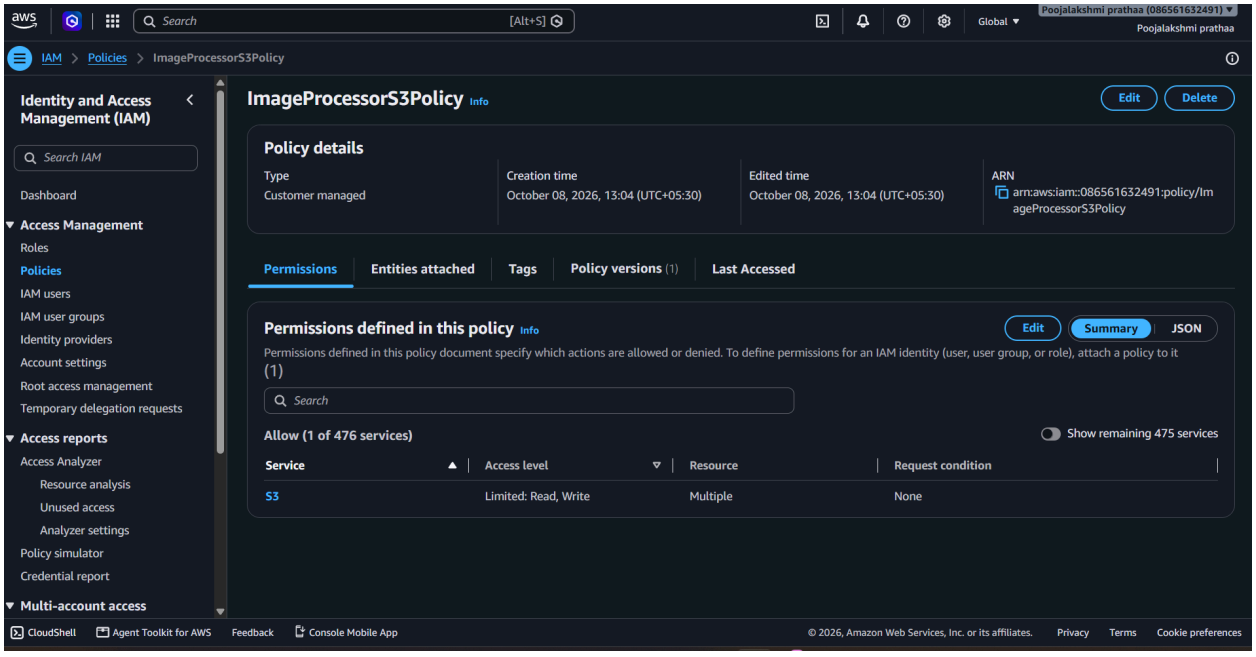


Figure 7. Custom S3 policy showing the restricted read and write permissions.

5. Recursive Invocation Problem and Prevention

A recursive invocation loop occurs when a Lambda function processes an S3 event, writes a new object to a bucket that triggers the same function, and repeatedly invokes itself. For example, if the function writes thumbnails to the same bucket that triggers it, each thumbnail may create another invocation. This can increase Lambda executions, S3 requests and AWS charges.

The implemented solution reduces this risk through the following controls:

1. **Separate output bucket:** Original images and generated thumbnails are stored in different buckets.
2. **Prefix filtering:** The S3 notification listens only for objects created under `incoming/`.
3. **AWS recursive loop detection:** Recursive loop detection is enabled in the Lambda configuration to help detect and terminate supported recursive invocation patterns.
4. **Suffix filtering:** An event notification can use supported suffix filters to restrict eligible file extensions, such as `.jpg` or `.png`. Suffix filtering alone is not a complete recursion-prevention method.
5. **Processed-object tracking:** Object tags or a processing record can identify already-processed inputs, provided the application or event-processing logic checks that information.

AWS recursion detection does not catch every possible loop. Therefore, separating input and output resources and filtering events remain important preventive controls.

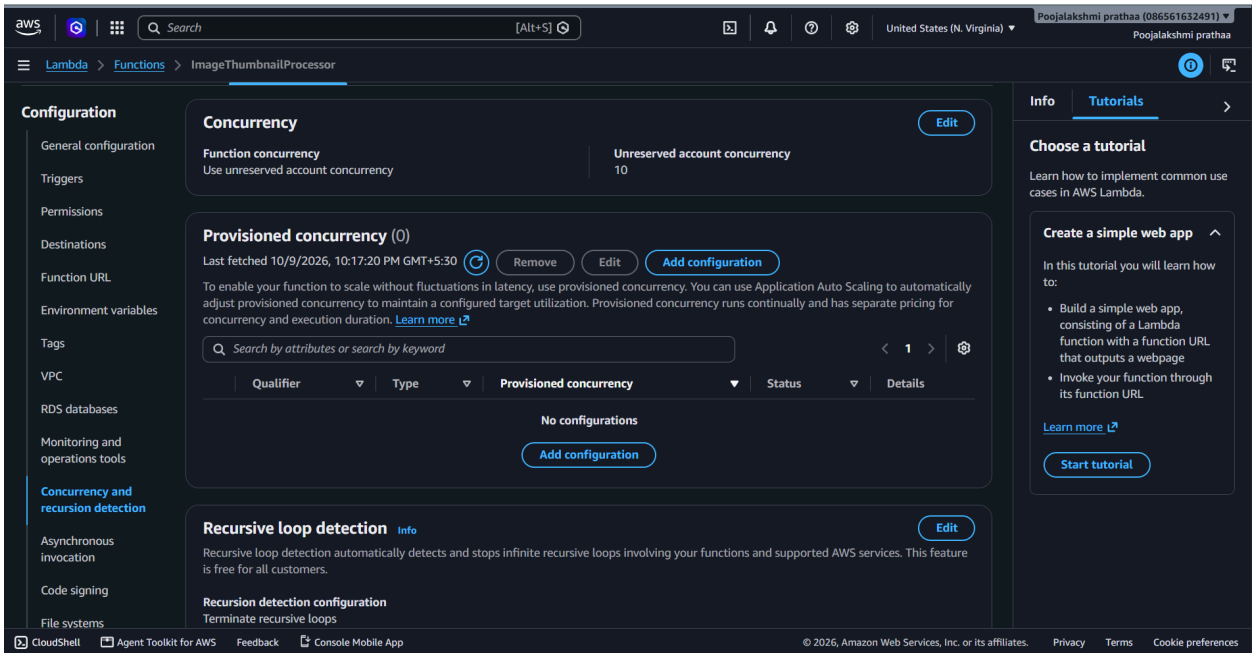


Figure 8. Lambda configuration showing recursive loop detection enabled.

6. CloudWatch Monitoring and Error Analysis

Amazon CloudWatch monitors Lambda execution metrics and stores application logs. The Lambda function prints processing messages and confirms the destination of each successfully created thumbnail.

The relevant metrics include:

- **Invocations:** Number of times the function runs.
- **Errors:** Number of failed executions.
- **Duration:** Time taken to process images.
- **Throttles:** Executions that could not proceed because of concurrency limits.
- **Recursive invocations:** Available recursion-related metrics, where data is reported.

During the observed successful test period, CloudWatch showed two invocations, zero errors and zero throttles. The function duration was approximately 250 milliseconds on average. No recursive-invocation metric data was available in the displayed time range; this does not establish that recursion detection was tested.

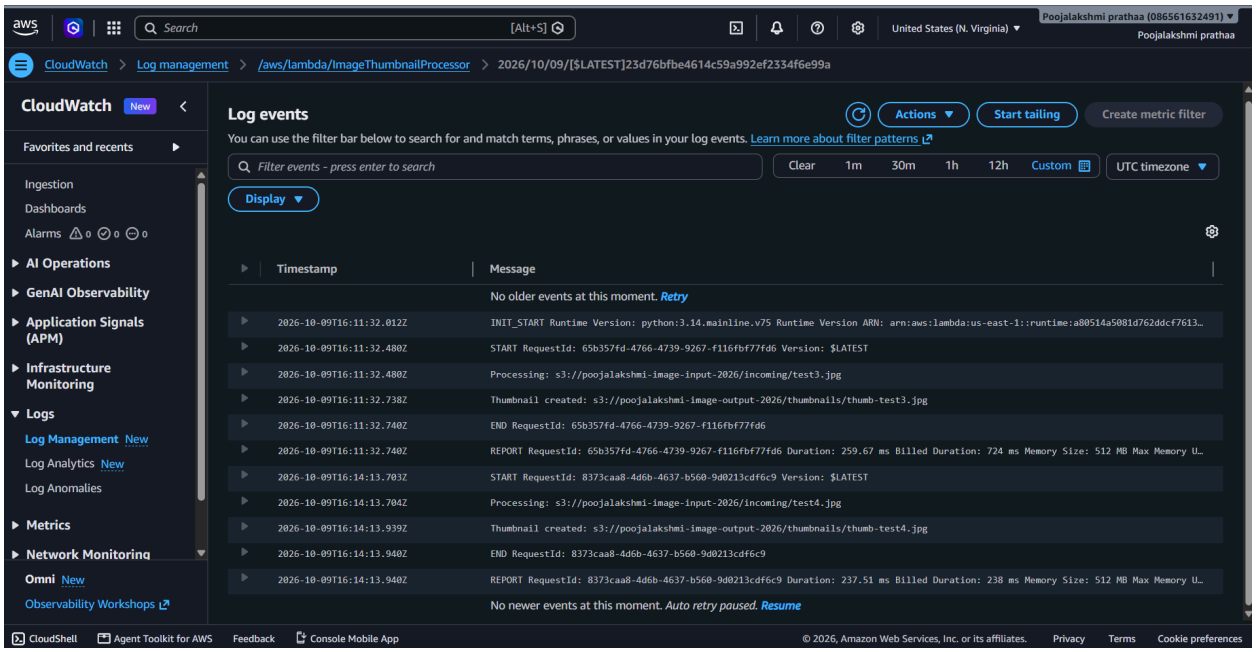


Figure 9. CloudWatch Lambda dashboard showing invocations, duration, errors, throttles and recursive-invocation metrics.

7. Emergency Shutdown and Cost Assessment

If an unexpected recursive loop occurs in production, the function must be stopped quickly to limit further executions and charges.

The emergency response procedure is:

1. Set Lambda reserved concurrency to zero to prevent new executions of the function.
2. Disable the S3 event trigger or the corresponding bucket notification to stop further event-driven invocations.
3. Review CloudWatch metrics and log streams to identify repeated invocations, failures and the likely cause.
4. Review AWS Cost Explorer for increased Lambda and S3 charges.
5. Review AWS Budgets and configure appropriate cost alerts.
6. Correct the configuration or code before restoring concurrency and re-enabling the trigger.

Reserved concurrency set to zero is an emergency brake, not a normal operating configuration. It was not activated during the successful thumbnail-processing test.

8. Idempotency and At-Least-Once Event Delivery

Amazon S3 event notifications use at-least-once delivery. Consequently, the same event can be delivered more than once, and the Lambda function may process an image repeatedly.

A robust idempotent design ensures that processing the same input event repeatedly does not cause unintended duplicate effects. A deterministic output key can prevent multiple differently named thumbnails from being created, but it does not by itself prevent repeated processing.

The implementation can be strengthened by tracking the input object version or another stable input identifier and recording processing status. A conditional write to a DynamoDB record can help prevent concurrent duplicate processing. The design must also handle retries and failures so that a failed operation can be recovered safely.

The simple thumbnail-existence check is a basic duplicate-event safeguard, not complete production-grade idempotency. If an input image is overwritten, the application must determine whether the new version requires a new thumbnail.

9. End-to-End Application Flow

The workflow follows these steps:

1. A user uploads an image to `poojalakshmi-image-input-2026/incoming/`.
2. Amazon S3 generates an ObjectCreated event.
3. The configured S3 notification invokes `ImageThumbnailProcessor`.

4. Lambda validates the image extension and downloads the input object.
5. Pillow resizes the image to fit within 200 × 200 pixels.
6. Lambda uploads the generated thumbnail to `poojalakshmi-image-output-2026/thumbnails/`.
7. CloudWatch records processing messages and execution metrics.
8. IAM permissions restrict the function's access to the required S3 resources and logging services.

The separate output bucket and input prefix filter prevent the normal output path from feeding back into the same trigger.

10. Security and Operational Assessment

Area	Implementation and assessment
Compute	Lambda processes uploaded images without requiring a continuously running server.
Storage	Separate input and output buckets reduce recursive-trigger risk.
Event filtering	The `incoming/` prefix limits eligible input objects.
IAM	Read and write permissions are restricted to the required object paths.
Monitoring	CloudWatch metrics and logs provide execution visibility.
Recursion protection	AWS recursive loop detection is enabled, with architectural prevention controls.
Idempotency	Duplicate processing requires reliable input-version or processing-status tracking for production use.
Cost management	CloudWatch, Cost Explorer and AWS Budgets can help identify and control unexpected usage.

11. Conclusion

The implemented AWS image-processing solution uses Amazon S3, AWS Lambda, Pillow, IAM and CloudWatch to generate thumbnails from uploaded images. The successful tests confirmed that supported images can be processed and stored in the output bucket. Separate storage buckets and an input-prefix filter help prevent recursive invocation, while AWS recursive loop detection provides an additional safeguard.

For reliable production operation, the solution should include robust idempotency, least-privilege access, monitoring, cost alerts and a documented emergency-stop procedure. Together, these controls improve the reliability, security and operational safety of the image-processing workflow.